2. Explain different concepts of Object Oriented Programming.

· Encapsulation: Binding data and methods together, hiding internal details.

· Inheritance: Creating new class from existing class.

· Polymorphism: Same function name with different implementations.

· Abstraction: Hiding complex details, showing only essential features.

3. Define Polymorphism. Write a Python program for polymorphism through inheritance.

· Polymorphism: Ability to take multiple forms (same method name works differently for different classes).

```python
class Animal:
    def sound(self):
        pass

class Dog(Animal):
    def sound(self):
        return "Bark"

class Cat(Animal):
    def sound(self):
        return "Meow"

# Polymorphism in action
```

```
animals = [Dog(), Cat()]

for a in animals:

    print(a.sound())
```

4. Define Abstraction. Write a Python program to understand the concept of abstract class.

· Abstraction: Hiding implementation details, showing only functionality.

```python
from abc import ABC, abstractmethod

class Shape(ABC):  # Abstract class

    @abstractmethod

    def area(self):

        pass

class Rectangle(Shape):

    def __init__(self, l, b):

        self.l = l

        self.b = b

    def area(self):

        return self.l * self.b

r = Rectangle(5, 3)
```

```
print("Area:", r.area())
```

5. Explain set data structure in Python.

· Set: Unordered collection of unique elements.

· Mutable, no duplicates, supports mathematical set operations.

· Example: s = {1, 2, 3}, operations: union, intersection, difference.

6. Define Tuple in Python. Explain operations of tuple data structure in Python.

· Tuple: Ordered, immutable sequence of elements.

· Operations: Indexing, slicing, concatenation, repetition, membership test.

· Example: t = (1, 2, 3), t[0] gives 1, t + (4,) gives (1, 2, 3, 4).

7. Explain types of constructors in Python. Write a Python program to multiply two numbers using static method.

· Types:

1. Default constructor: No parameters
2. Parameterized constructor: With parameters
3. Non-parameterized constructor: Explicitly defined without parameters

```python
class MathOps:
    @staticmethod
```

```
    def multiply(a, b):
        return a * b

# Using static method
result = MathOps.multiply(5, 4)
print("Multiplication:", result)
```

8. Define Data Encapsulation. List different types of methods in Python. Write a Python program for multilevel inheritance.

· Data Encapsulation: Wrapping data and methods together, restricting direct access.

· Method types: Instance, Class, Static methods.

```python
class GrandParent:
    def show_gp(self):
        print("GrandParent class")

class Parent(GrandParent):
    def show_parent(self):
        print("Parent class")

class Child(Parent):
    def show_child(self):
        print("Child class")
```

```
c = Child()
c.show_gp()     # From GrandParent
c.show_parent()  # From Parent
c.show_child()   # From Child
```

Q2 — Algorithm Analysis, Stack & Queue

9. Define the following terms:

· Best case: Minimum time taken by algorithm (ideal input)

· Worst case: Maximum time taken (least ideal input)

· Average case: Expected time for random input

10. Explain infix, postfix and prefix expressions.

· Infix: Operator between operands: A + B

· Postfix: Operator after operands: A B +

· Prefix: Operator before operands: + A B

11. Define circular queue. Explain INSERT and DELETE operations of circular queue with diagrams.

· Circular queue: Last element points to first, forming circle.

· INSERT (enqueue): Add element at rear, if full → overflow.

· DELETE (dequeue): Remove from front, if empty → underflow.

· (Diagram: Circular array with front/rear pointers moving circularly)

12. List different Data Structures with examples.

· Linear: Array, Linked List, Stack, Queue

· Non-Linear: Tree, Graph

· Hash-based: Hash Table

· File-based: Sequential, Indexed files

13. Differentiate between simple queue and circular queue.

· Simple queue: Linear, once full cannot reuse space.

· Circular queue: Circular, reuses empty spaces efficiently.

14. Define stack. Explain PUSH and POP operation with example. Write algorithm for PUSH and POP operation of stack.

· Stack: LIFO (Last In First Out) structure.

PUSH Algorithm:

```

1. Check if stack is full (if top == max_size-1)
```

2. If full, display "Overflow"

3. Else, increment top

4. Insert element at top position

```

POP Algorithm:

```

1. Check if stack is empty (if top == -1)

2. If empty, display "Underflow"

3. Else, remove element from top

4. Decrement top

```

15. Explain disadvantages of queue.

· Fixed size in array implementation

· Wasted space in linear queue

· Not suitable for frequent insertion/deletion in middle

16. Define abstract class in Python. Explain declaration of abstract method in Python.

· Abstract class: Cannot instantiate, meant for inheritance.

· Abstract method: Declared but not implemented in base class.

```python

```
from abc import ABC, abstractmethod

class AbstractClass(ABC):
    @abstractmethod
    def abstract_method(self):
        pass
```

17. Write an algorithm for infix to postfix expression. Evaluate postfix expression.

Infix to Postfix Algorithm:

```
1. Initialize empty stack and output string
2. Scan infix expression left to right
3. If operand → add to output
4. If '(' → push to stack
5. If ')' → pop until '('
6. If operator → pop higher/equal precedence operators, then push current
7. End expression → pop all from stack to output
```

Postfix Evaluation Algorithm:

```
1. Initialize empty stack
```

2. Scan postfix left to right

3. If operand → push to stack

4. If operator → pop two operands, apply operator, push result

5. End → stack top is result

```

---

Q3 — Linked List

18. Convert the following infix expression to postfix: ((A-B)*C)+((D-E)/F)

```

Step-by-step conversion:

((A-B)*C)+((D-E)/F)

1. (A-B)*C → A B - C *

2. (D-E)/F → D E - F /

3. Combine: A B - C * D E - F / +

Final: A B - C * D E - F / +

```

19. Write a short note on doubly linked list.

· Each node has data, next pointer, and previous pointer.

· Allows traversal in both directions.

· More memory but flexible operations.
```

20. Write a Python program to delete first and last node from singly linked list.

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def delete_first(self):
        if self.head:
            self.head = self.head.next

    def delete_last(self):
        if not self.head:
            return
        if not self.head.next:
            self.head = None
            return
        temp = self.head
        while temp.next.next:
            temp = temp.next
```

```
        temp.next = None

# Usage
ll = LinkedList()
# ... add nodes ...
ll.delete_first()
ll.delete_last()
```

21. List applications of queue.

· CPU Scheduling

· Printer Spooling

· Call Center systems

· Breadth-First Search (BFS)

22. Explain different operations performed on singly linked list.

· Insertion: Beginning, End, Specific position

· Deletion: Beginning, End, Specific position

· Traversal

· Searching

· Reversal

23. Write an algorithm to insert a new node at the end of doubly linked list.

```

1. Create new node with data

2. If list empty → make new node as head

3. Else traverse to last node

4. Set last_node.next = new_node

5. Set new_node.prev = last_node

6. Set new_node.next = None
```

24. Write an algorithm to traverse nodes in singly linked list.

```

1. Start from head node

2. While current node is not None:

3. Process current node data

4. Move to next node (current = current.next)
```

25. Write an algorithm for dequeue operation in queue using list.

```

1. Check if queue is empty (front > rear or front == -1)

2. If empty → display "Queue Underflow"

3. Else:

4. Remove element from front position

5. Increment front by 1
```

6. If front > rear → reset queue to empty

```

26. Define doubly linked list. Enlist major operations of linked list. Write an algorithm to insert a node at beginning in singly linked list.

· Doubly linked list: Each node has next and previous pointers.

Major operations: Insertion, Deletion, Traversal, Searching, Reversal

Insert at beginning (Singly LL) algorithm:

```

1. Create new node with data

2. Set new_node.next = current_head

3. Set head = new_node

```

27. Explain applications of singly linked list.

· Dynamic memory allocation

· Implementation of stacks/queues

· Polynomial representation

· Music playlist

28. Write an algorithm for PUSH operation of stack using list.
```

```

1. Check if stack is full (top == max_size-1)

2. If full → display "Stack Overflow"

3. Else:

4. Increment top by 1

5. stack[top] = element

```

29. Explain advantages of linked list. Write an algorithm to delete last node from singly linked list.

· Advantages: Dynamic size, efficient insertion/deletion, no memory waste

Delete last node algorithm:

```

1. If list empty → return

2. If only one node → head = None

3. Else:

4. Traverse to second last node

5. Set second_last.next = None

```

Q4 — Searching & Sorting

30. Write a Python program for linear search.

```python
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1

arr = [10, 20, 30, 40, 50]
result = linear_search(arr, 30)
print("Element found at index:", result)
```

31. Write a short note on circular linked list.

· Last node points to first node, forming circle.

· Can be singly or doubly circular.

· Useful for round-robin scheduling.

32. Explain quick sort algorithm with an example.

· Quick Sort: Divide and conquer, picks pivot, partitions array.

· Example: [10, 80, 30, 90, 40, 50, 70]

· Pivot = 70

· Partition: [10, 30, 40, 50] 70 [80, 90]

· Recursively sort subarrays

33. Explain binary search algorithm with an example.

· Binary Search: Works on sorted array, repeatedly divides in half.

· Example: Search 23 in [10, 20, 23, 30, 40, 50]

· Mid = 23 → found

34. Discuss applications of linked list.

· Implementation of stacks/queues

· Dynamic memory management

· Undo functionality in software

· Hash table chaining

35. Write a Python program for insertion sort with example.

```python
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and key < arr[j]:
            arr[j + 1] = arr[j]
```

```
        j -= 1
    arr[j + 1] = key
  return arr

arr = [12, 11, 13, 5, 6]
print("Sorted:", insertion_sort(arr))
```

36. Write an algorithm of bubble sort.

```
1. For i from 0 to n-1:
2.    For j from 0 to n-i-2:
3.       If arr[j] > arr[j+1]:
4.          Swap arr[j] and arr[j+1]
```

37. Explain circular linked list with its advantages.

· Advantages: Efficient round-robin, no null pointers at end, easy circular traversal.

38. Explain merge sort with suitable example.

· Merge Sort: Divide array, recursively sort, merge.

· Example: [38, 27, 43, 3, 9, 82, 10]

 · Divide into single elements

· Merge sorted pairs: [27,38] [3,43] [9,82] [10]

· Continue merging...

39. Write an algorithm for selection sort.

```
1. For i from 0 to n-2:
2.     min_index = i
3.     For j from i+1 to n-1:
4.         If arr[j] < arr[min_index]:
5.             min_index = j
6.     Swap arr[i] and arr[min_index]
```

40. Explain doubly linked list with its advantages.

· Advantages: Bidirectional traversal, easier deletion of previous node, more flexible.

41. Explain insertion sort. Give trace of following numbers using insertion sort.

· Insertion Sort: Builds sorted array one element at a time.

Trace for [5, 2, 4, 6, 1, 3]:

```
Pass 1: [5|2,4,6,1,3] → [2,5|4,6,1,3]
```

Pass 2: [2,5,4|6,1,3] → [2,4,5|6,1,3]

Pass 3: [2,4,5,6|1,3] → [2,4,5,6|1,3]

Pass 4: [2,4,5,6,1|3] → [1,2,4,5,6|3]

Pass 5: [1,2,4,5,6,3] → [1,2,3,4,5,6]

```

---

Q5 — Trees

42. Define the following terms:

· Complete Binary Tree: All levels filled except possibly last, filled left to right.

· In-degree: Number of edges coming into a node (in trees, root has 0, others 1).

· Out-degree: Number of edges going out from a node.

43. Explain bubble sort algorithm with an example.

· Repeatedly swaps adjacent elements if in wrong order.

· Example: [5, 1, 4, 2, 8]

  · Pass 1: [1, 5, 4, 2, 8] → [1, 4, 5, 2, 8] → [1, 4, 2, 5, 8]

  · Pass 2: [1, 4, 2, 5, 8] → [1, 2, 4, 5, 8] (sorted)

44. Create a Binary Search Tree for given keys and write Preorder, Inorder and Postorder traversal.
```

· Keys: 50, 30, 70, 20, 40, 60, 80

· BST:

```
      50
     / \
    30  70
   / \  / \
  20 40 60 80
```

· Preorder: 50, 30, 20, 40, 70, 60, 80

· Inorder: 20, 30, 40, 50, 60, 70, 80

· Postorder: 20, 40, 30, 60, 80, 70, 50

45. Define binary tree. Explain searching a node in binary tree.

· Binary tree: Each node has at most 2 children (left, right).

· Searching: Start from root, compare key, go left if smaller, right if larger (for BST).

46. Give the trace to sort the given data using bubble sort method.

Data:[64, 34, 25, 12, 22, 11, 90]

```
Pass 1: [34,64,25,12,22,11,90] → [34,25,64,12,22,11,90] → [34,25,12,64,22,11,90]
→ [34,25,12,22,64,11,90] → [34,25,12,22,11,64,90]
```

Pass 2: [25,34,12,22,11,64,90] → [25,12,34,22,11,64,90] → [25,12,22,34,11,64,90]

→ [25,12,22,11,34,64,90]

...

Final: [11,12,22,25,34,64,90]

```

47. List applications of trees. Explain technique for converting general tree into binary search tree with example.

· Applications: File systems, DNS, Decision trees, Database indexing.

· Conversion: Use left-child, right-sibling representation.

48. Explain application of binary tree.

· Expression trees for compilers

· Huffman coding for compression

· Binary search trees for searching

· Heap for priority queues

49. Write an algorithm for binary search using list.

```

1. Set low = 0, high = n-1

2. While low <= high:

3. mid = (low + high) // 2

4. If arr[mid] == key → return mid

5. Else if arr[mid] < key → low = mid + 1

6. Else → high = mid - 1

7. Return -1 (not found)

```

50. Define tree. Enlist types of tree. Write an algorithm to insert node in binary search tree using Python.

· Tree: Hierarchical structure with nodes and edges.

· Types: Binary, BST, AVL, B-tree, Heap, Trie.

```python
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None

def insert(root, key):
    if root is None:
        return Node(key)
    if key < root.key:
        root.left = insert(root.left, key)
    else:
        root.right = insert(root.right, key)
    return root
```
```

```

51. Write an algorithm for inorder traversal of tree.

```

Inorder(node):

1. If node is not None:
2. Inorder(node.left)
3. Visit(node.data)
4. Inorder(node.right)
```

52. Define search. Write an algorithm for linear search using list.

· Search: Finding location of element in data structure.

Linear Search Algorithm:

```

1. For i from 0 to n-1:
2. If arr[i] == key:
3. Return i
4. Return -1
```

53. Define:
```

· Path: Sequence of nodes from root to a particular node.

· Leaf Node: Node with no children.

54. Construct a binary search tree for given data items.

Data:8, 3, 10, 1, 6, 14, 4, 7, 13

```
```

BST:

```
      8
     / \
    3   10
   / \   \
  1   6   14
     / \  /
    4   7 13
```

---

1. Differentiate between Linear and Non-Linear Data Structure:

1) Linear Data Structure	Non-Linear Data Structure
1) Elements arranged sequentially	Elements arranged hierarchically/randomly
2) Single level	
2) Multiple levels	

3)Traversal is linear (one after another)

3) Traversal is non-linear (multiple paths)

4) Examples: Array, Linked List, Stack, Queue

4) Examples: Tree, Graph

5) Memory efficient for sequential access

5)Better for complex relationships

6)Easier to implement

6) More complex implementation

---

13. Differentiate between simple queue and circular queue:

1)Simple Queue

1) Circular Queue

2)Linear arrangement

2) Circular arrangement

3) Wasted space after deletions
3) Reuses empty spaces

4) Fixed size, cannot reuse deleted space
4) Efficient memory utilization

5)Front & rear move only forward

5)Front & rear move circularly

6)Overflow when rear == max_size-1

6) Overflow when (rear+1) % max_size == front

7) Simple implementation Slightly

7) complex implementation

8) Example: Normal waiting line

8) Example: CPU scheduling

---

26. Differentiate between Singly and Doubly Linked List:

1)Singly Linked List

1) Doubly Linked List

2) Each node has data + next pointer

2) Each node has data + next + previous pointers

3) Traversal only forward

3) Traversal both forward and backward

5) Less memory per node (1 pointer)
5) More memory per node (2 pointers)

6) Deletion of previous node requires traversal
6) Easy deletion of previous node

7) Simpler implementation
7) More complex implementation

8)Cannot access previous node

8) directly Can access previous node directly

9)Used when backward traversal not needed

9)Used when bidirectional traversal needed

Differentiation between Searching Algorithms (Linear vs Binary):

1) Linear Search
2) Binary Search

- Works on any array (sorted/unsorted)
- Requires sorted array

-Sequential checking

-Divide and conquer approach

-Time: O(n)

-Time: O(log n)

-Simple to implement

-More complex implementation

-No preprocessing needed

-Array must be sorted first

-Suitable for small datasets

-Suitable for large datasets

Differentiation between Stack and Queue:

1)Stack

2) Queue

-LIFO (Last In First Out)

- FIFO (First In First Out)

-One end operations (top)

-Two end operations (front/rear)

-Used for recursion, undo operations

- Used for scheduling, BFS

-Operations: Push, Pop

- Operations: Enqueue, Dequeue

-Example: Plate stacking

- Example: Ticket counter line

Differentiation between Array and Linked List:

1) Array
2) Linked List

-Contiguous memory

-Non-contiguous memory

-Fixed size

-Dynamic size

-Random access (O(1))

-Sequential access (O(n))

-Insertion/deletion costly

- Insertion/deletion efficient

-Memory efficient (no pointers)

- Extra memory for pointers

-Cache friendly

-Cache unfriendly

-Suitable for frequent access

-Suitable for frequent modifications
